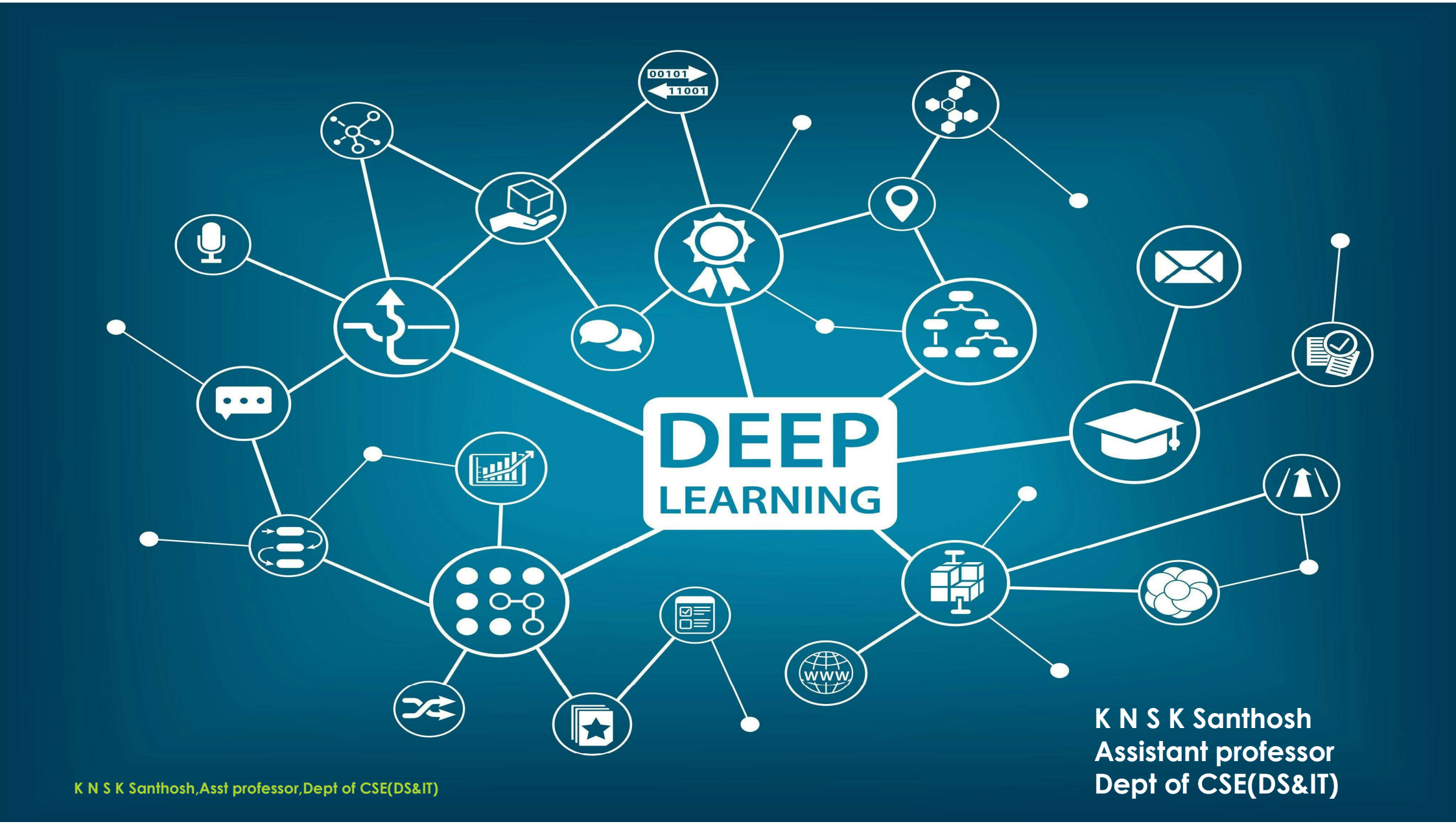
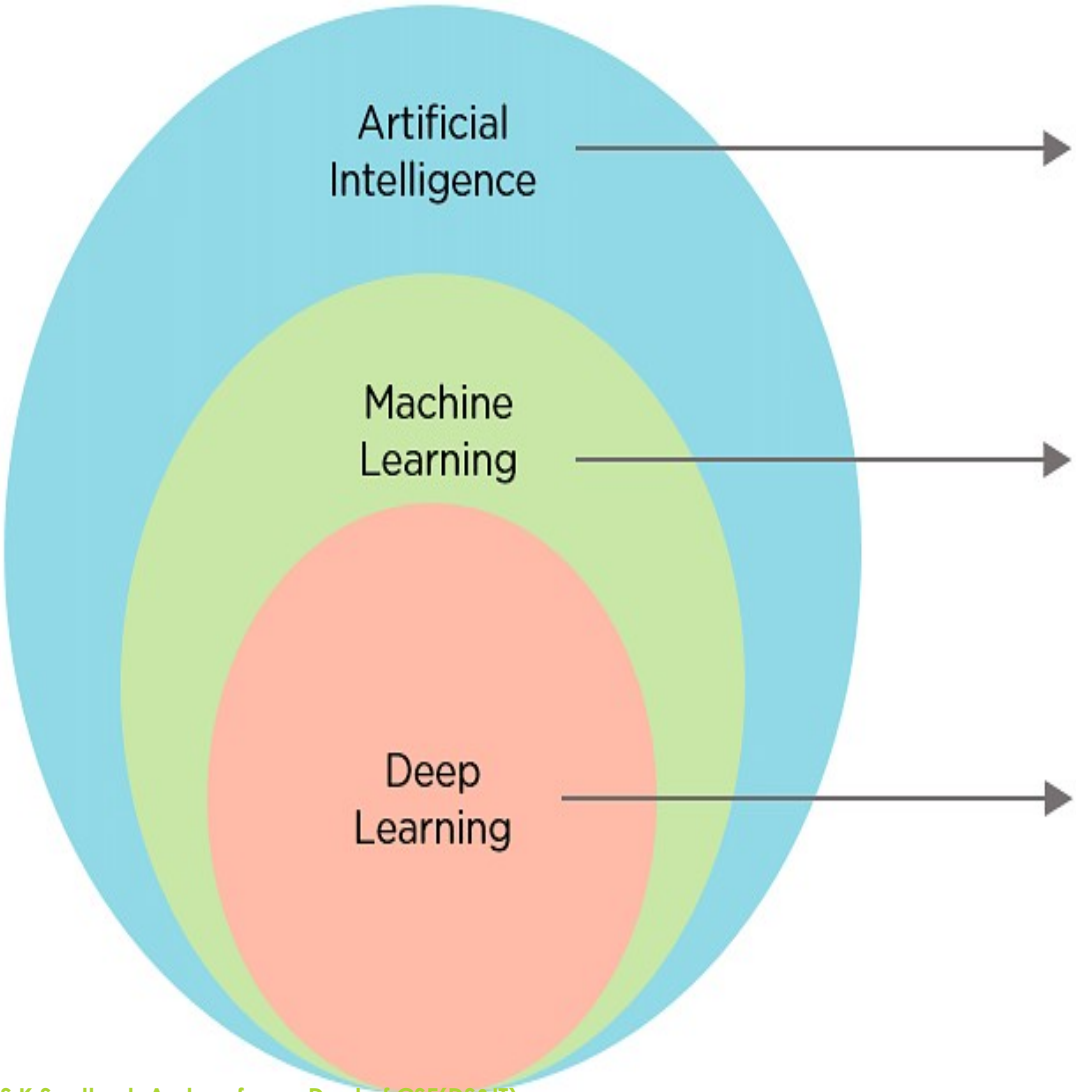


K N S K Santhosh
Assistant professor
Dept of CSE(DS&IT)

UNIT-I SYLLABUS

Fundamentals of Deep Learning: Artificial Intelligence, History of Machine learning: Probabilistic Modeling, Early Neural Networks, Kernel Methods, Decision Trees, Random forests and Gradient Boosting Machines

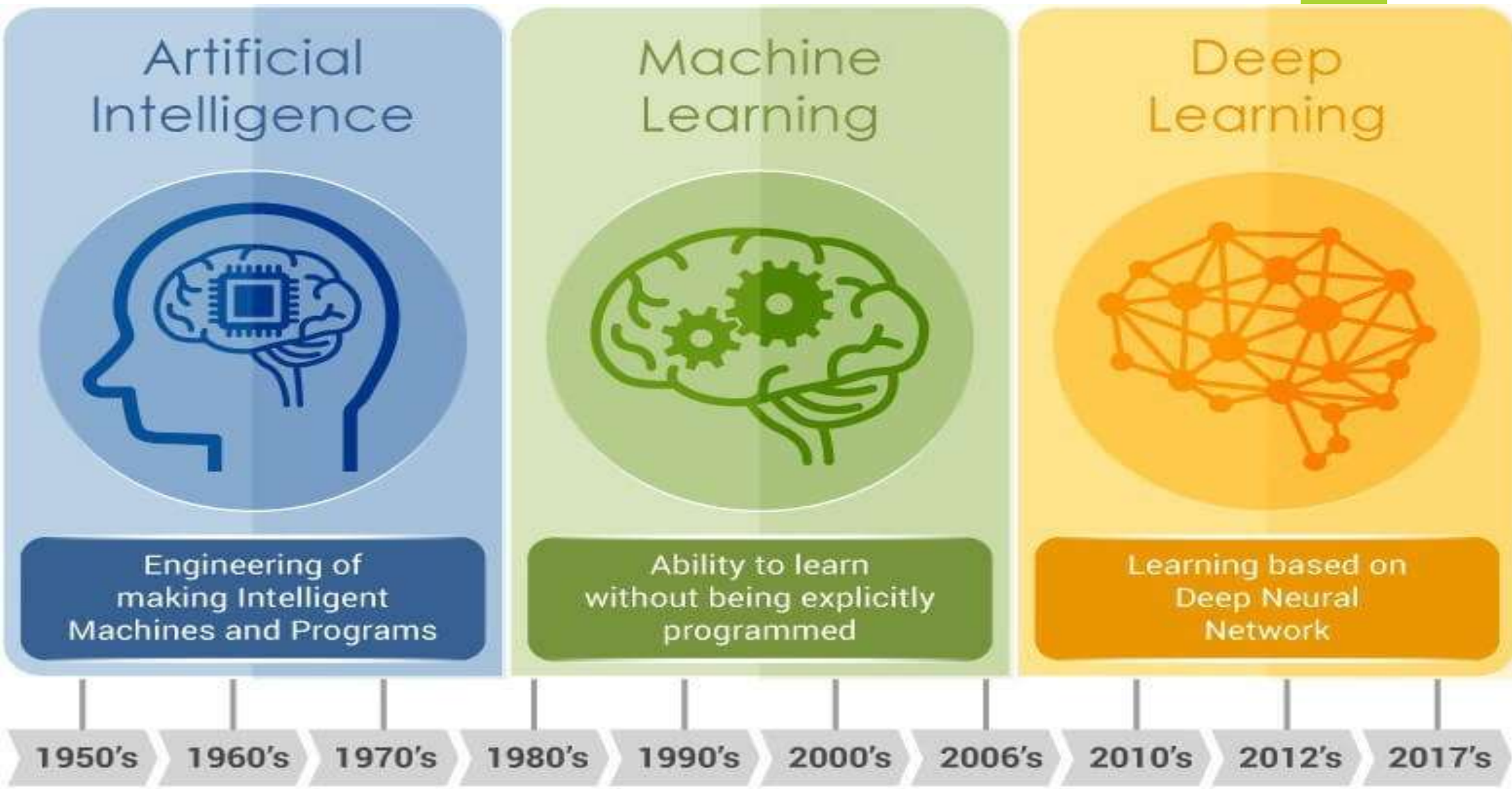
Fundamentals of Machine Learning: Four Branches of Machine Learning, Evaluating Machine learning Models, Overfitting and Underfitting



Ability of a machine to imitate intelligent human behavior

Application of AI that allows a system to automatically learn and improve from experience

Application of Machine Learning that uses complex algorithms and deep neural nets to train a model


Artificial Intelligence
Machine Learning
Deep Learning
Engineering of making Intelligent Machines and Programs
Ability to learn without being explicitly programmed
Learning based on Deep Neural Network
1950's
1960's
1970's
1980's
1990's
2000's
2006's
2010's
2012's
2017's

Artificial intelligence

- Artificial intelligence was born in the 1950s, when a handful of pioneers from the nascent field of computer science started asking whether computers could be made to “think”—a question whose ramifications we’re still exploring today.



Artificial intelligence

- A concise definition of the field would be as follows: the effort to automate intellectual tasks normally performed by humans. As such, AI is a general field that encompasses machine learning and deep learning, but that also includes many more approaches that don't involve any learning.
- Early chess programs, for instance, only involved hardcoded rules crafted by programmers, and didn't qualify as machine learning.

Artificial intelligence

- This approach is known as symbolic AI, and it was the dominant paradigm in AI from the 1950s to the late 1980s. It reached its peak popularity during the expert systems boom of the 1980s
- Symbolic AI has been applied in various fields, including natural language processing, expert systems, and robotics. Some specific examples include: Siri and other digital assistants use Symbolic AI to understand natural language and provide responses

Machine learning

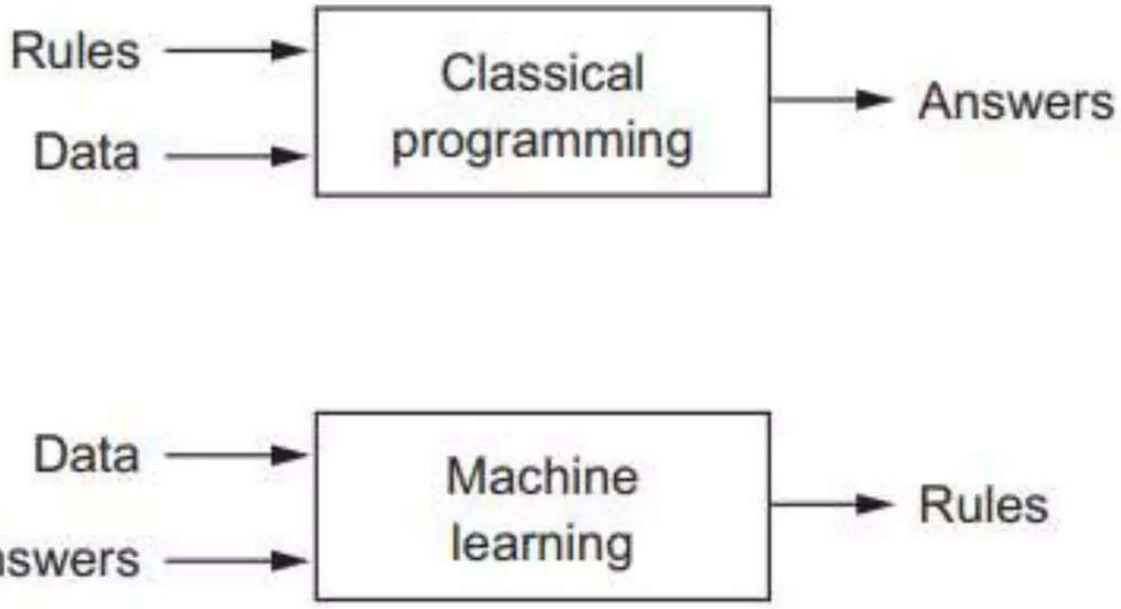
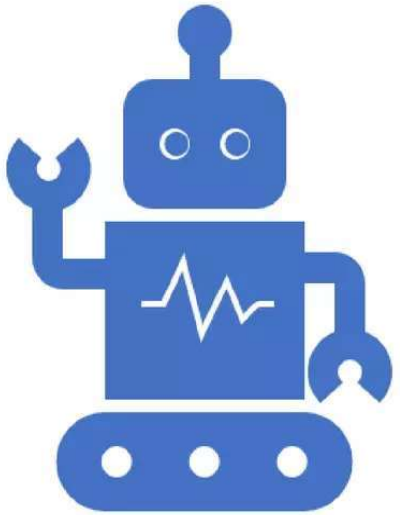


Figure 1.2 Machine learning: a new programming paradigm

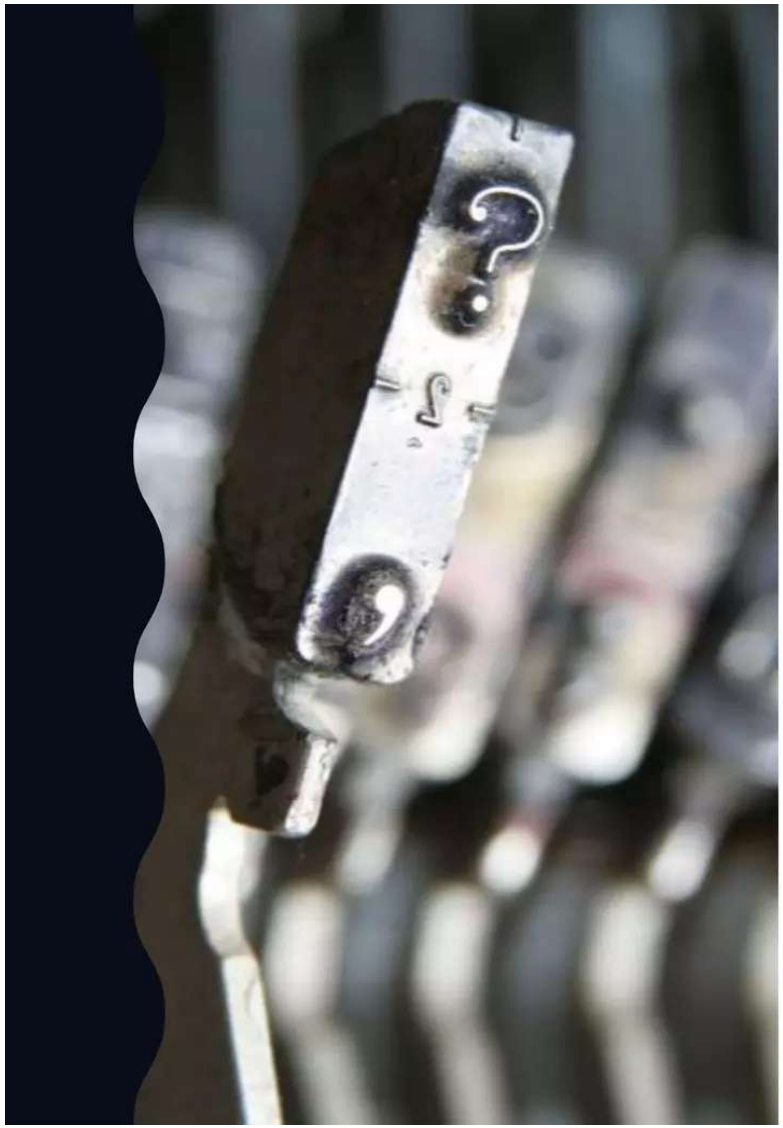
Machine Learning

- A machine-learning system is trained rather than explicitly programmed. It's presented with many examples relevant to a task, and it finds statistical structure in these examples that eventually allows the system to come up with rules for automating the task.
- For instance, if you wished to automate the task of tagging your vacation pictures, you could present a machine-learning system with many examples of pictures already tagged by humans, and the system would learn statistical rules for associating specific pictures to specific tags.



Artificial Intelligence	Machine Learning	Deep Learning
AI stands for Artificial Intelligence, and is basically the study/process which enables machines to mimic human behaviour through particular algorithm.	ML stands for Machine Learning, and is the study that uses statistical methods enabling machines to improve with experience.	DL stands for Deep Learning, and is the study that makes use of Neural Networks(similar to neurons present in human brain) to imitate functionality just like a human brain.
AI is the broader family consisting of ML and DL as it's components.	ML is the subset of AI.	DL is the subset of ML.
AI is a computer algorithm which exhibits intelligence through decision making.	ML is an AI algorithm which allows system to learn from data.	DL is a ML algorithm that uses deep(more than one layer) neural networks to analyze data and provide output accordingly.
Search Trees and much complex math is involved in AI.	If you have a clear idea about the logic(math) involved in behind and you can visualize the complex functionalities like K-Mean, Support Vector Machines, etc., then it defines the ML aspect.	If you are clear about the math involved in it but don't have idea about the features, so you break the complex functionalities into linear/lower dimension features by adding more layers, then it defines the DL aspect.

Artificial Intelligence	Machine Learning	Deep Learning
Three broad categories/types Of AI are: Artificial Narrow Intelligence (ANI), Artificial General Intelligence (AGI) and Artificial Super Intelligence (ASI)	Three broad categories/types Of ML are: Supervised Learning, Unsupervised Learning and Reinforcement Learning	DL can be considered as neural networks with a large number of parameters layers lying in one of the four fundamental network architectures: Unsupervised Pre-trained Networks, Convolutional Neural Networks, Recurrent Neural Networks and Recursive Neural Networks
The efficiency Of AI is basically the efficiency provided by ML and DL respectively.	Less efficient than DL as it can't work for longer dimensions or higher amount of data.	More powerful than ML as it can easily work for larger sets of data.
AI can be further broken down into various subfields such as robotics, natural language processing, computer vision, expert systems, and more.	ML algorithms can be categorized as supervised, unsupervised, or reinforcement learning. In supervised learning, the algorithm is trained on labeled data, where the desired output is known. In unsupervised learning, the algorithm is trained on unlabeled data, where the desired output is unknown.	DL algorithms are inspired by the structure and function of the human brain, and they are particularly well-suited to tasks such as image and speech recognition.
The aim is to basically increase chances of success and not accuracy.	The aim is to increase accuracy not caring much about the success ratio.	It attains the highest rank in terms of accuracy when it is trained with large amount of data.



Machine Learning and Statistics

- Although machine learning only started to flourish in the 1990s, it has quickly become the most popular and most successful subfield of AI, a trend driven by the availability of faster hardware and larger datasets.
- Machine learning is tightly related to mathematical statistics, but it differs from statistics in several important ways.
- Unlike statistics, machine learning tends to deal with large, complex datasets (such as a dataset of millions of images, each consisting of tens of thousands of pixels) for which classical statistical analysis such as Bayesian analysis would be impractical.
- As a result, machine learning, and especially deep learning, exhibits comparatively little mathematical theory—maybe too little—and is engineering oriented. It's a hands-on discipline in which ideas are proven empirically more often than theoretically.

Needed For Machine Learning

- *Input data points*—For instance, if the task is speech recognition, these data points could be sound files of people speaking. If the task is image tagging, they could be pictures.
- *Examples of the expected output*—In a speech-recognition task, these could be human-generated transcripts of sound files. In an image task, expected outputs could be tags such as "dog," "cat," and so on.
- *A way to measure whether the algorithm is doing a good job*—This is necessary in order to determine the distance between the algorithm's current output and its expected output. The measurement is used as a feedback signal to adjust the way the algorithm works. This adjustment step is what we call *learning*.

Representation of data in ML

Let's make this concrete. Consider an x-axis, a y-axis, and some points represented by their coordinates in the (x, y) system, as shown in figure 1.3.

As you can see, we have a few white points and a few black points. Let's say we want to develop an algorithm that can take the coordinates (x, y) of a point and output whether that point is likely to be black or to be white. In this case,

- The inputs are the coordinates of our points.
- The expected outputs are the colors of our points.
- A way to measure whether our algorithm is doing a good job could be, for instance, the percentage of points that are being correctly classified.

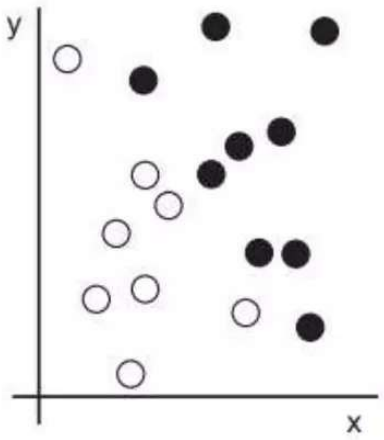


Figure 1.3
Some sample data

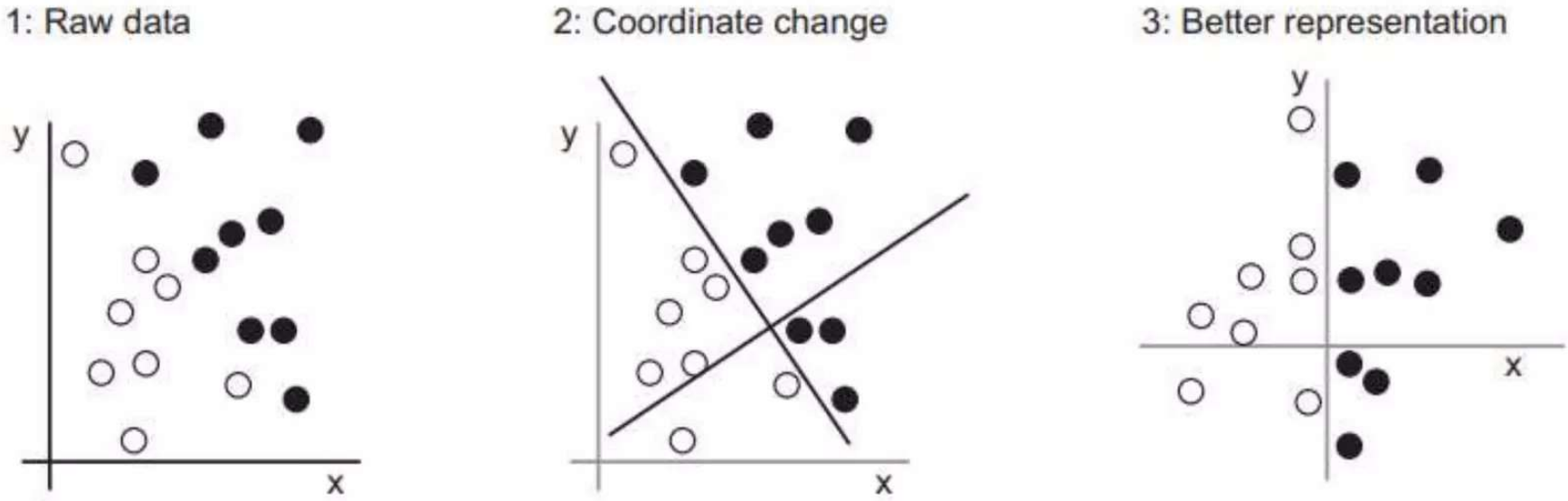


Figure 1.4 Coordinate change



Representation

- Learning, in the context of machine learning, describes an automatic search process for better representations.
- All machine-learning algorithms consist of automatically finding such transformations that turn data into more-useful representations for a given task.
- These operations can be coordinate changes, as you just saw, or linear projections (which may destroy information), translations, nonlinear operations (such as “select all points such that x > 0”), and so on.

What makes Deep Learning Special?



- The deep in deep learning isn't a reference to any kind of deeper understanding achieved by the approach; rather, it stands for this idea of successive layers of representations.
- How many layers contribute to a model of the data is called the depth of the model.
- Other appropriate names for the field could have been layered representations learning and hierarchical representations learning.
- Modern deep learning often involves tens or even hundreds of successive layers of representations— and they're all learned automatically from exposure to training data.

What is shallow learning?

- Meanwhile, other approaches to machine learning tend to focus on learning only one or two layers of representations of the data; hence, they're sometimes called shallow learning.

- Imagine you are trying to understand a book. You could start by reading the first page, then the second page, and so on. This would be like shallow learning. You would only be able to understand the book at a superficial level.
- However, you could also read the book by first reading the introduction, then the table of contents, then the chapters in order. This would be like deep learning. You would be able to understand the book at a much deeper level

Deep learning

In deep learning, these layered representations are (almost always) learned via models called neural networks, structured in literal layers stacked on top of each other.

There's no evidence that the brain implements anything like the learning mechanisms used in modern deep-learning models. You may come across pop-science articles proclaiming that deep learning works like the brain or was modeled after the brain, but that isn't the case.

Deep Learning in simple words

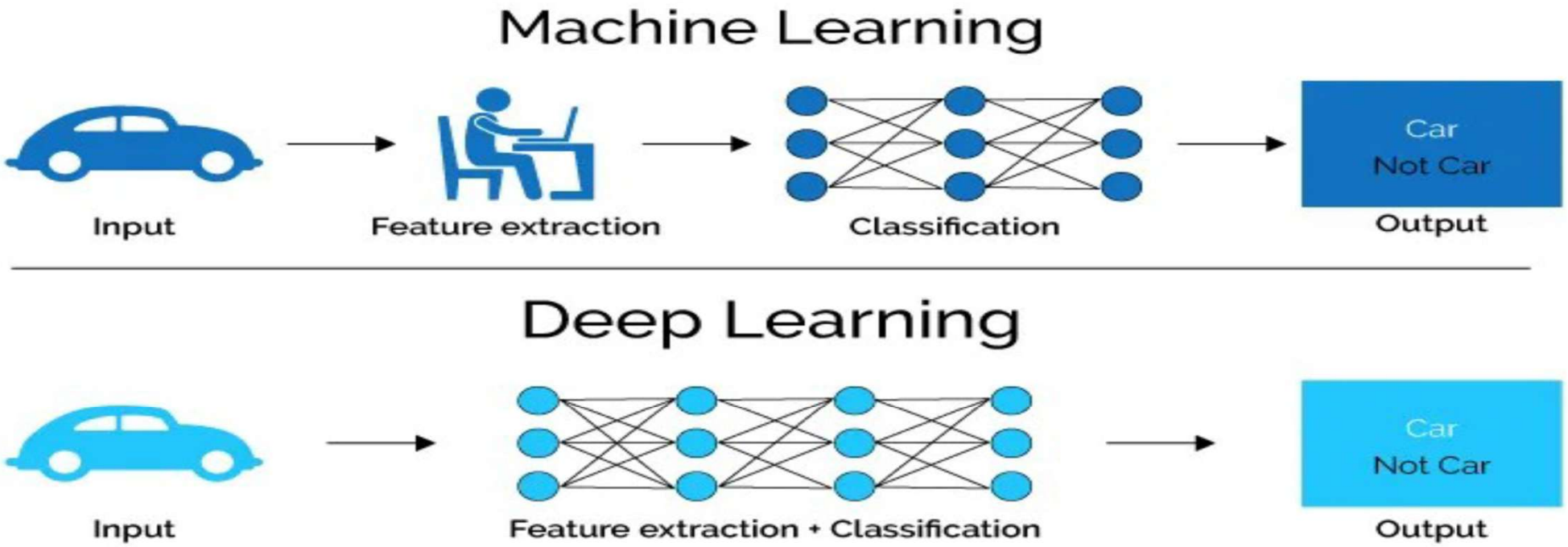
Deep learning is a mathematical framework for learning representations from data.

K N S K Santhosh,Asst professor,Dept of CSE(DS&IT)

The "deep" in deep learning

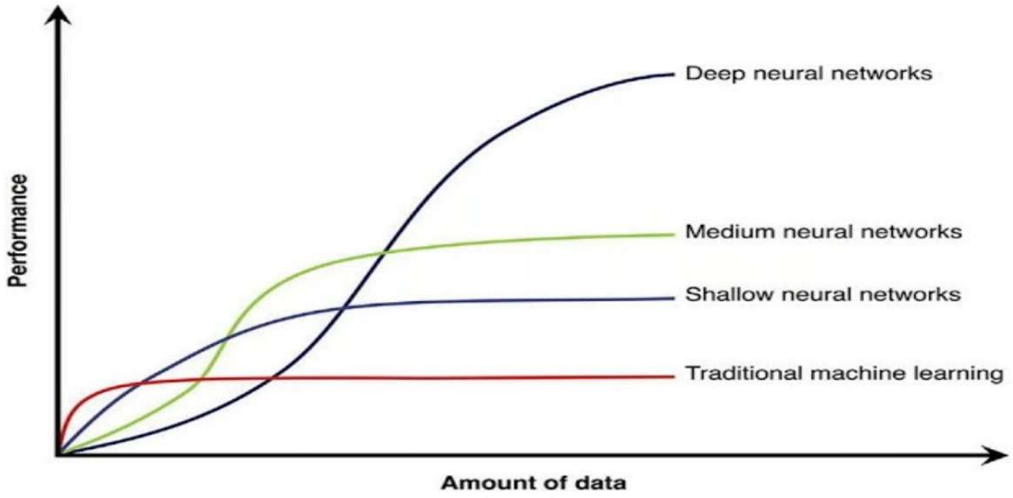
- Deep learning is a type of machine learning that learns from data by creating successive layers of increasingly meaningful representations. The depth of a deep learning model refers to how many layers it has. Modern deep learning models often have tens or even hundreds of layers.
- In contrast, other approaches to machine learning typically only learn one or two layers of representations. These approaches are sometimes called shallow learning.
- The main difference between deep learning and shallow learning is that deep learning models can learn more complex representations of data. This makes them more powerful and able to solve more difficult problems.
- Here is an analogy that might help you understand deep learning:
- Imagine you are trying to understand a book. You could start by reading the first page, then the second page, and so on. This would be like shallow learning. You would only be able to understand the book at a superficial level.
- However, you could also read the book by first reading the introduction, then the table of contents, then the chapters in order. This would be like deep learning. You would be able to understand the book at a much deeper level

Deep Learning vs. Machine Learning



Why Deep Learning ?

- When the amount of data is increased, machine learning techniques are insufficient in terms of performance and deep learning gives better performance like accuracy.



Why Now?

- *Algorithm Advancements*
- *GPU Computing*
- *Availability of Larger Training Data*

Deep Learning vs. Machine Learning

- Deep Learning can essentially do everything that machine learning does, but not the other way around.
- For instance, machine learning is useful when the dataset is small and well-curated, which means that the data is carefully preprocessed.
- Data preprocessing requires human intervention. It also means that when the dataset is large and complex, machine learning algorithms will fail to extract information, and it will underfit.
- Generally, machine learning is alternatively termed shallow learning because it is very effective for smaller datasets.
- Deep learning, on the other hand, is extremely powerful when the dataset is large.
- It can learn any complex patterns from the data and can draw accurate conclusions on its own. In fact, deep learning is so powerful that it can even process unstructured data - data that is not adequately arranged like text corpus, social media activity, etc.
- Furthermore, it can also generate new data samples and find anomalies that machine learning algorithms and human eyes can miss.

- On the downside, deep learning is computationally expensive compared to machine learning, which also means that it requires a lot of time to process.
- Deep Learning and Machine Learning are both capable of different types of learning: Supervised Learning (labeled data), Unsupervised Learning (unlabeled data), and Reinforcement Learning.
- But their usefulness is usually determined by the size and complexity of the data.

What do the representations learned by a deep-learning algorithm look like?

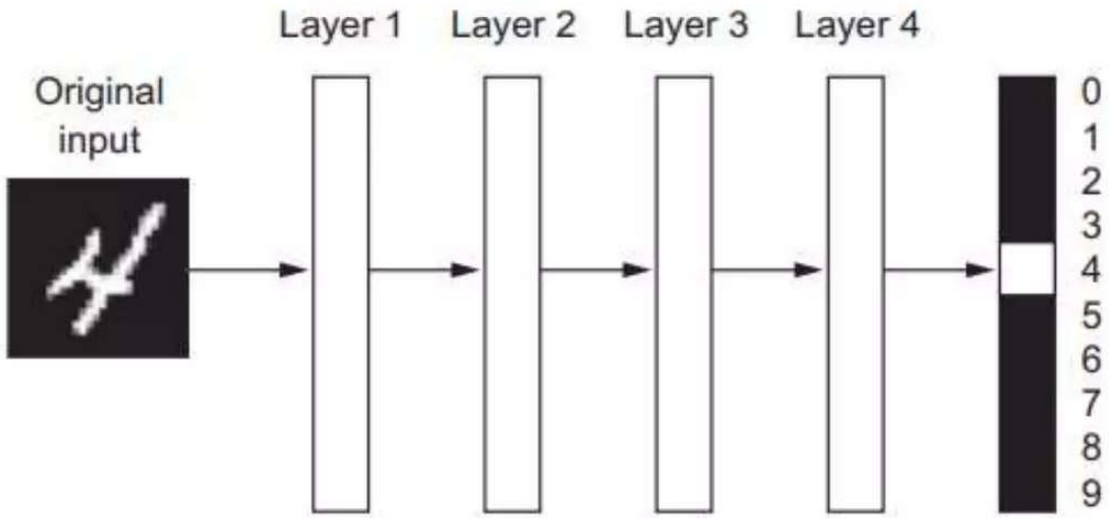


Figure 1.5 A deep neural network for digit classification

If the number of layered representations which need to be learnt are one or two, the learning is called as **shallow learning** and the model is termed as **shallow neural network.** In case, the large number of representations need to be learnt, the learning is called as **deep learning** and the model is called as **deep neural network.** The **deep learning**, at times, is also termed as **layered representations learning** or **hierarchical representations learning.**

You can think of a deep network as a multistage information-distillation operation, where information goes through successive filters and comes out increasingly purified (that is, useful with regard to some task)

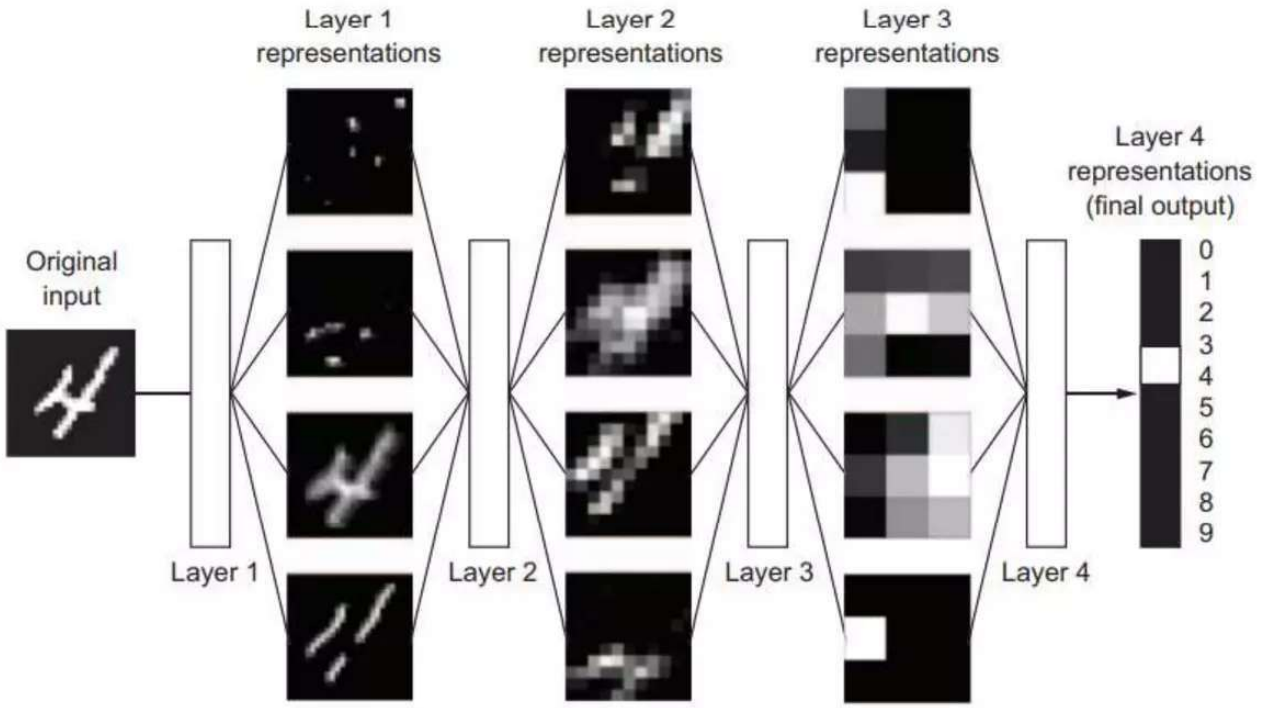
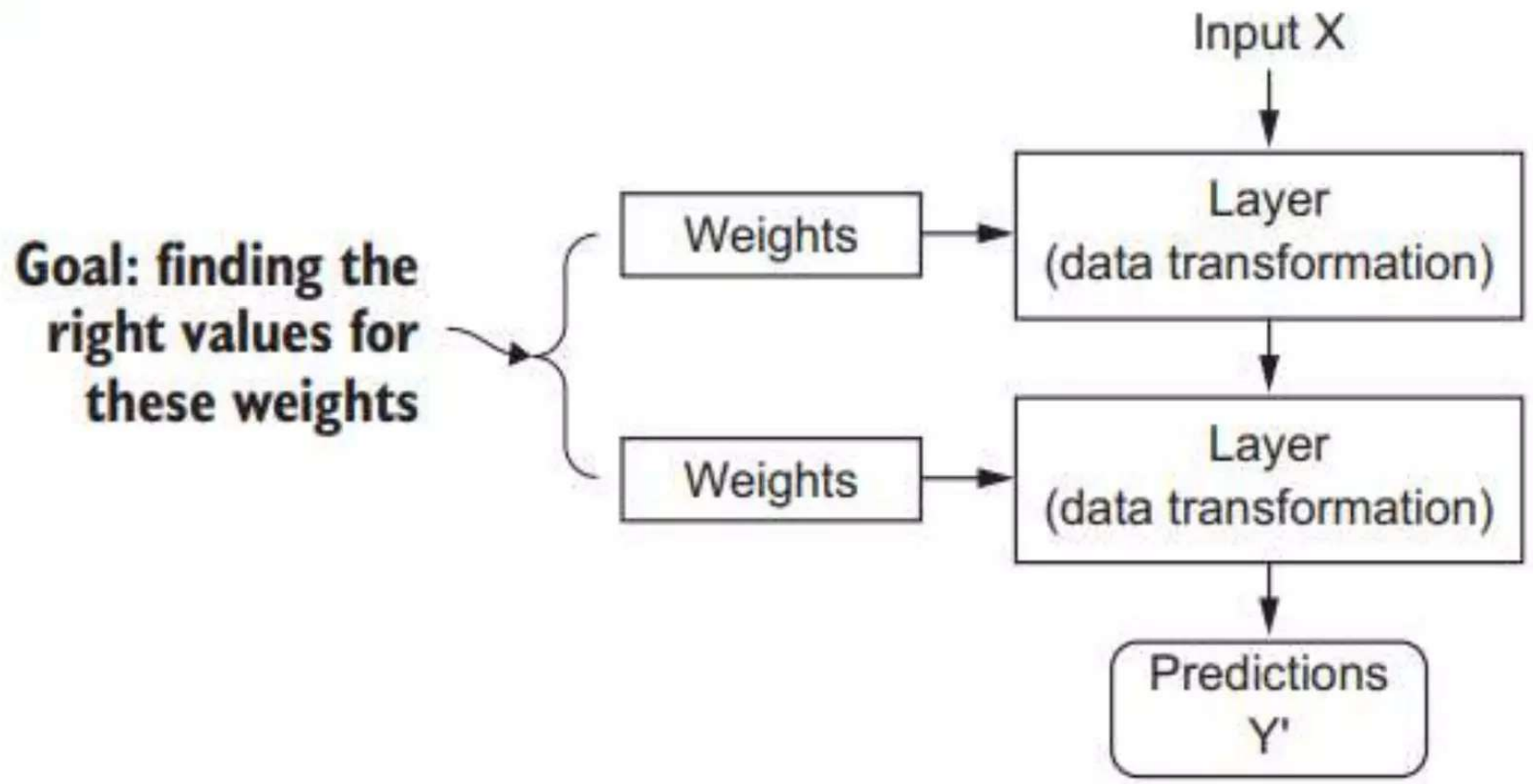


Figure 1.6 Deep representations learned by a digit-classification model


Input X
Goal: finding the right values for these weights
Weights
Weights
Layer (data transformation)
Layer (data transformation)
Predictions Y'

Finding right values for these weights

The specification of what a layer does to its input data is stored in the layer's weights, which in essence are a bunch of numbers. In technical terms, we'd say that the transformation implemented by a layer is parameterized by its weights (see figure in previous slide).

(Weights are also sometimes called the parameters of a layer.) In this context, learning means finding a set of values for the weights of all layers in a network, such that the network will correctly map example inputs to their associated targets.

But here's the thing: a deep neural network can contain tens of millions of parameters. Finding the correct value for all of them may seem like a daunting task, especially given that modifying the value of one parameter will affect the behavior of all the

Loss Function

To control something, first you need to be able to observe it. To control the output of a neural network, you need to be able to measure how far this output is from what you expected.

This is the job of the loss function of the network, also called the objective function. The loss function takes the predictions of the network and the true target (what you wanted the network to output) and computes a distance score, capturing how well the network has done on this specific example (See next Figure).

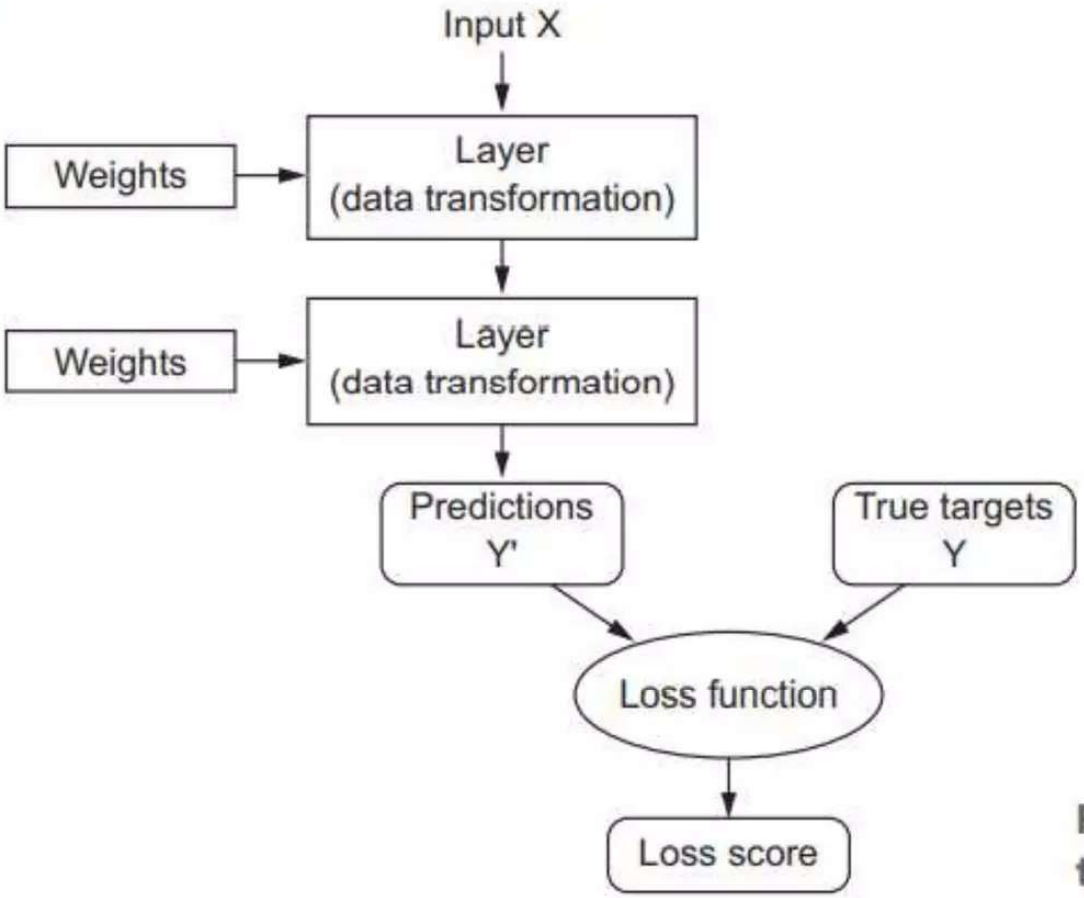


Figure 1.8 A loss function measures the quality of the network's output.

BackPropagation Algorithm

The fundamental trick in deep learning is to use this score as a feedback signal to adjust the value of the weights a little, in a direction that will lower the loss score for the current example (see figure 1.9).

This adjustment is the job of the optimizer, which implements what's called the Backpropagation algorithm: the central algorithm in deep learning.

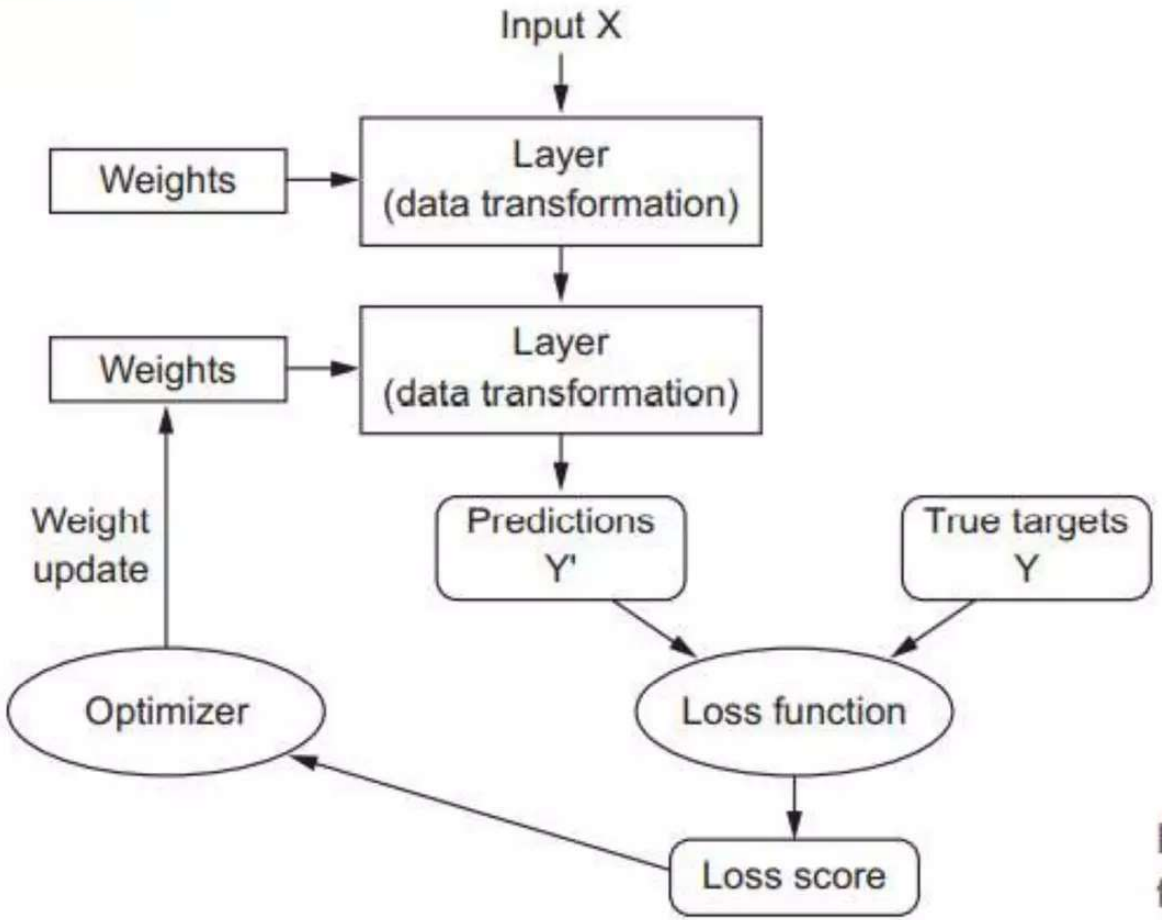


Figure 1.9 The loss score is used as a feedback signal to adjust the weights.

Method



Initially, the weights of the network are assigned random values, so the network merely implements a series of random transformations. Naturally, its output is far from what it should ideally be, and the loss score is accordingly very high.



But with every example the network processes, the weights are adjusted a little in the correct direction, and the loss score decreases. This is the training loop, which, repeated a sufficient number of times (typically tens of iterations over thousands of examples), yields weight values that minimize the loss function.



A network with a minimal loss is one for which the outputs are as close as they can be to the targets: a trained network.



Once again, it's a simple mechanism that, once scaled, ends up looking like magic.

What deep learning has Achieved so far?

Near-human-level image classification	Near-human-level speech recognition	Near-human-level handwriting transcription	Improved machine Translation
Improved text to speech conversion	digital assistants such as Google Now and Amazon Alexa	improve the search results on the web	ability to answer natural language questions

Deep Learning hype

Deep learning has reached a level of public attention and industry investment never before seen in the history of AI, but it isn't the first successful form of machine learning.

It's safe to say that most of the machine-learning algorithms used in the industry today aren't deep-learning algorithms.

Deep learning isn't always the right tool for the job—sometimes there isn't enough data for deep learning to be applicable, and sometimes the problem is better solved by a different algorithm.

If deep learning is your first contact with machine learning, then you may find yourself in a situation where all you have is the deep-learning hammer, and every machine-learning problem starts to look like a nail. The only way not to fall into this trap is to be familiar with other approaches and practice them when appropriate.

Before deep learning: a brief history of machine learning

- Deep learning has become very popular and is getting a lot of attention and investment in the AI field. However, it's essential to know that it's not the only successful type of machine learning. Many other algorithms are used in the industry today, and they are not necessarily deep learning algorithms.
- If you're new to machine learning and only know about deep learning, you might end up trying to use it for everything and see every problem as a "nail" for your "hammer."
- To avoid this trap, it's crucial to learn about other approaches to machine learning and use them when they are more suitable for the task at hand.

1. Probabilistic modelling
2. Early neural networks
3. Kernel Methods
4. Decision trees, random forests, and gradient boosting machines

Probabilistic modelling

Probabilistic modeling is a way to use statistics to understand data. It's been around for a long time, and it's still used a lot today. One of the most popular probabilistic models is called Naive Bayes.

Naive Bayes is a simple but powerful algorithm that can be used for a variety of tasks, including spam filtering, text classification, and medical diagnosis. It works by assuming that each feature in a dataset is independent of the others. This means that the probability of a certain outcome is only affected by the probability of that outcome for that feature.

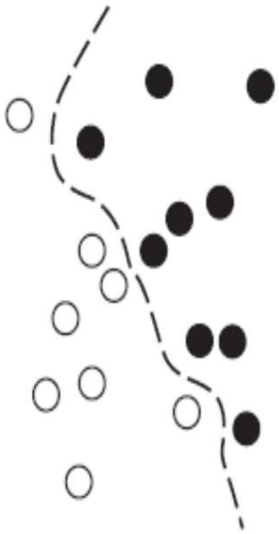
Naive Bayes is a type of machine learning algorithm that uses Bayes' theorem to classify data. It assumes that the features in the data are independent of each other, which is why it's called Naive Bayes.

Early neural networks

- Early neural networks were inspired by the biological neural networks in the human brain. They were simple models that consisted of a few layers of interconnected neurons. These neurons were typically threshold gates, which means that they would only activate if their input was above a certain threshold.
- The first early neural network was the **perceptron**, which was developed by **Frank Rosenblatt** in **1957**. The perceptron was a simple model that could be used to classify **binary data**. It was a single-layer neural network with a threshold gate at the output.
- Other early neural networks included the ADALINE and MADALINE networks, which were developed by **Bernard Widrow** and **Marcian Hoff** in **1959**. These networks were also single-layer neural networks, but they used a different type of neuron called a **linear unit**.

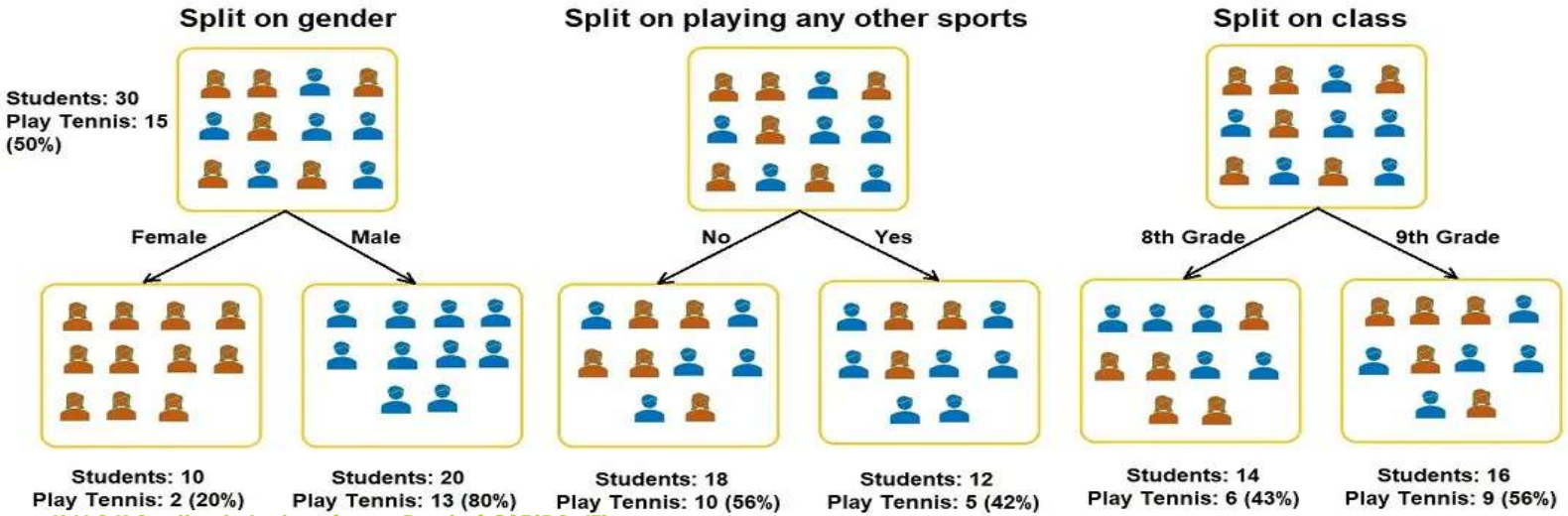
Kernel Methods

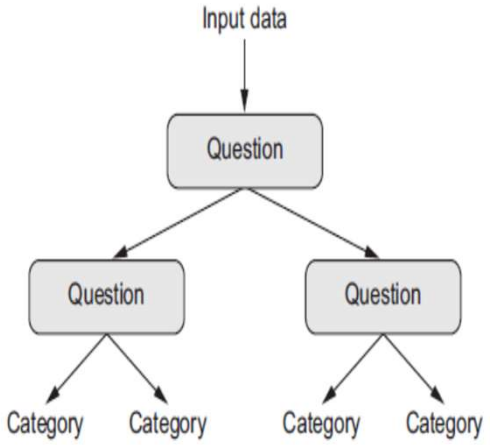
- As neural networks started to gain some respect among researchers in the 1990s, thanks to this first success, a new approach to machine learning rose to fame and quickly sent neural nets back to oblivion: kernel methods
- Kernel methods are a group of classification algorithms, the best known of which is the **support vector machine** (SVM).
- SVMs aim at solving classification problems by finding good decision boundaries between two sets of points belonging to two different categories.
- A decision boundary can be thought of as a line or surface separating your training data into two spaces corresponding to two categories.
- To classify new data points, you just need to check which side of the decision boundary they fall on.



Decision trees

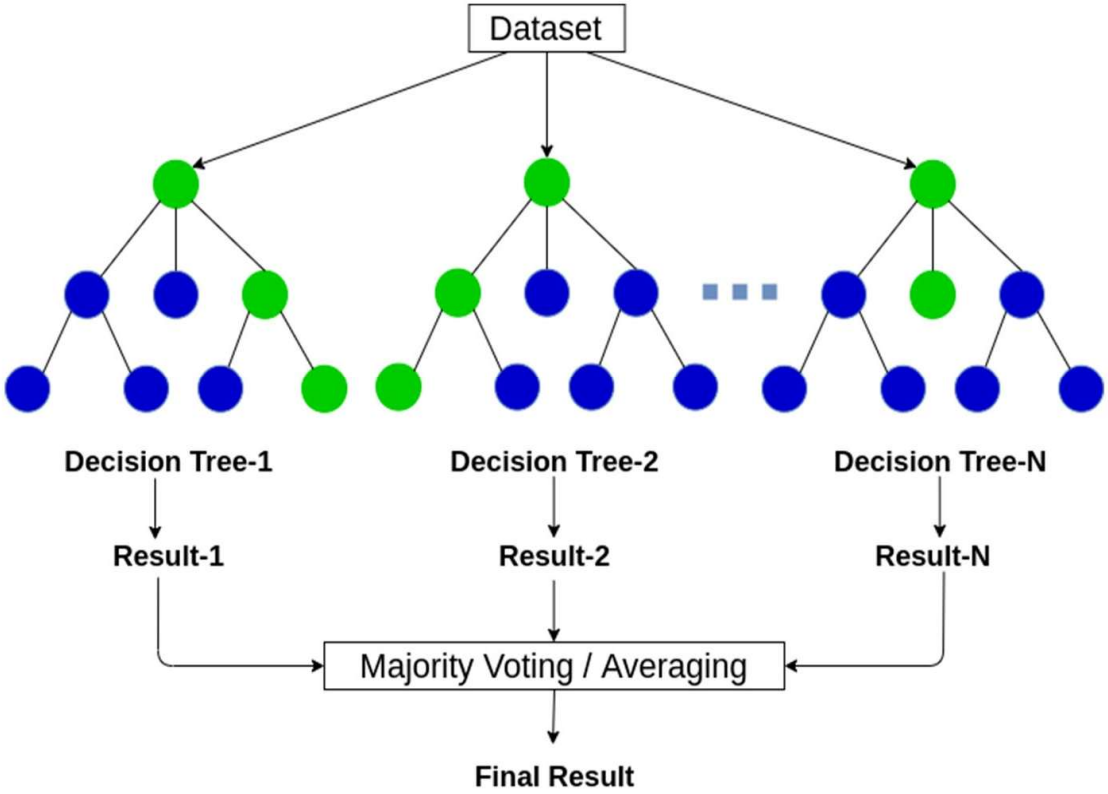
- Decision trees are flowchart-like structures that let you classify input data points or predict output values given inputs . They're easy to visualize and interpret.
- Decisions trees learned from data began to receive significant research interest in the 2000s, and by 2010 they were often preferred to kernel methods.





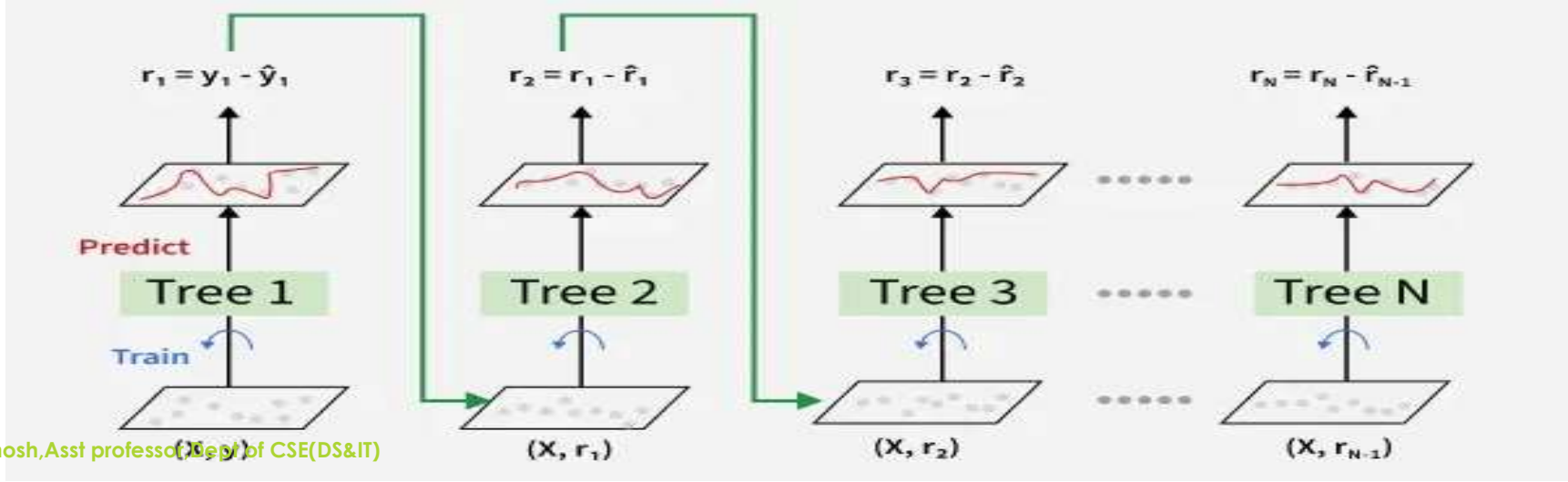
random forests

- In particular, the Random Forest algorithm introduced a robust, practical take on decision-tree learning that involves building a large number of specialized decision trees and then ensembling their outputs.
- Random forests are applicable to a wide range of problems—you could say that they're almost always the second-best algorithm for any shallow machine-learning task.



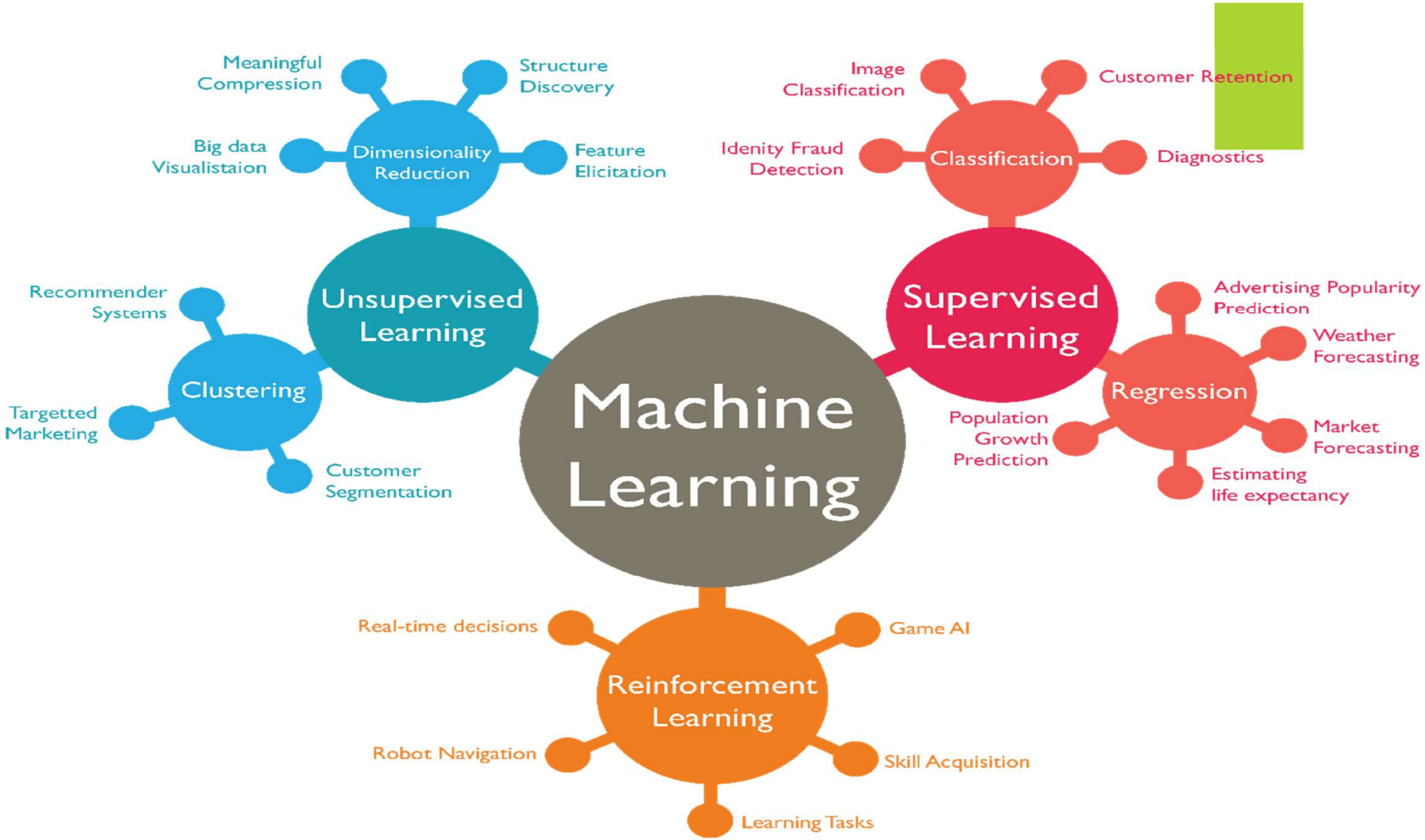
gradient boosting machines

A gradient boosting machine, much like a random forest, is a machine-learning technique based on ensembling weak prediction models, generally decision trees. It uses gradient boosting, a way to improve any machine-learning model by iteratively training new models that specialize in addressing the weak points of the previous models.



Four branches of machine learning

Branch	Description	Examples
Supervised learning	The model is trained on labeled data.	Spam filtering, image classification, fraud detection
Unsupervised learning	The model is trained on unlabeled data.	Clustering, dimensionality reduction, anomaly detection
Semi-supervised learning	The model is trained on a combination of labeled and unlabeled data.	Natural language processing, medical diagnosis
Reinforcement learning	The model learns to behave in an environment by trial and error.	Game playing, robotics


Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Market Forecasting
Estimating life expectancy
Population Growth Prediction
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

Evaluating Machine learning Models

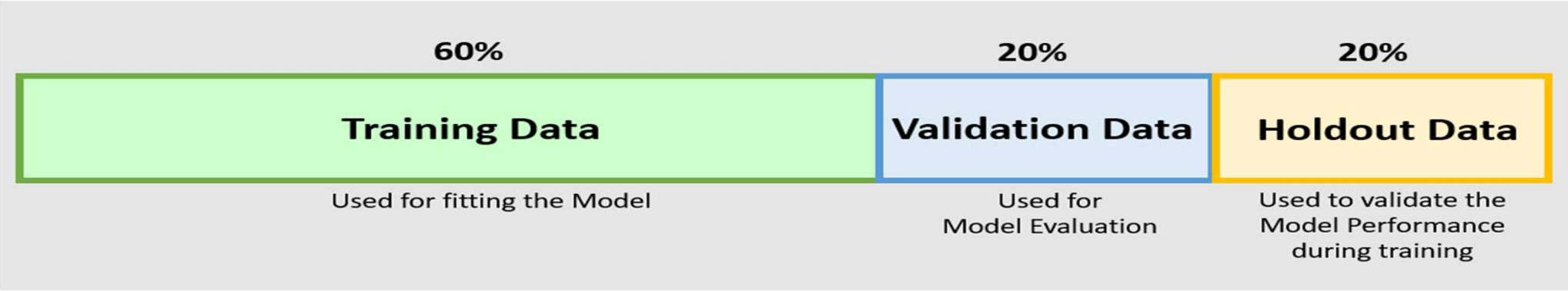
Training, validation, and test sets

- Evaluating a model always boils down to splitting the available data into three sets: training, validation, and test.
- You may ask, why not have two sets: a training set and a test set? You would train on the training data and evaluate on the test data. Much simpler!
  - **Overfitting**. If we only train on the training set, and then evaluate on the test set, there is a risk that the model will overfit the training data.
  - **Bias**. If we only use the training set to evaluate the model, we may not be able to identify any biases in the model.
  - **Unreliability**. If we only use the test set to evaluate the model, then the results of the evaluation may not be reliable.
- To address these issues, we often split the data into three sets: a training set, a validation set, and a test set.

The training set is used to train the model, the validation set is used to tune the hyperparameters of the model, and the test set is used to evaluate the final model.

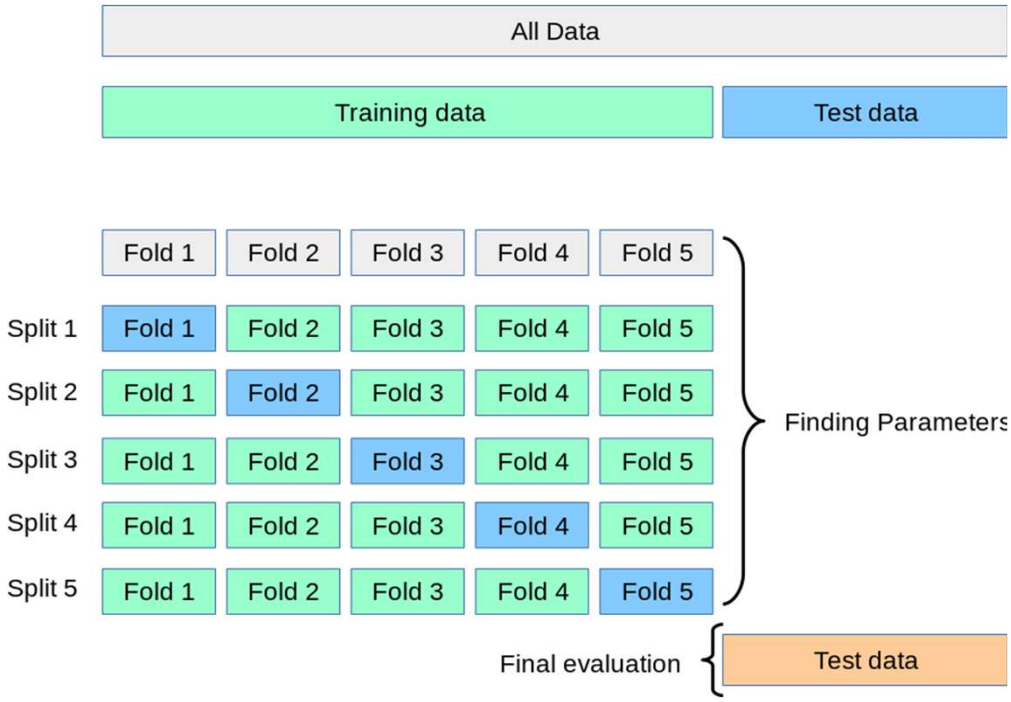
1. SIMPLE HOLD-OUT VALIDATION

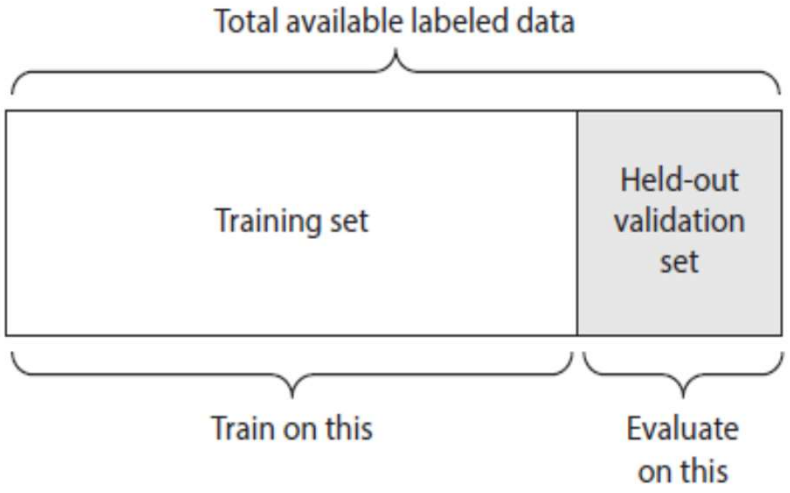
Simple hold-out validation is a method of evaluating a machine learning model by splitting the data into two sets: a training set and a test set. The training set is used to train the model, and the test set is used to evaluate the model's performance.

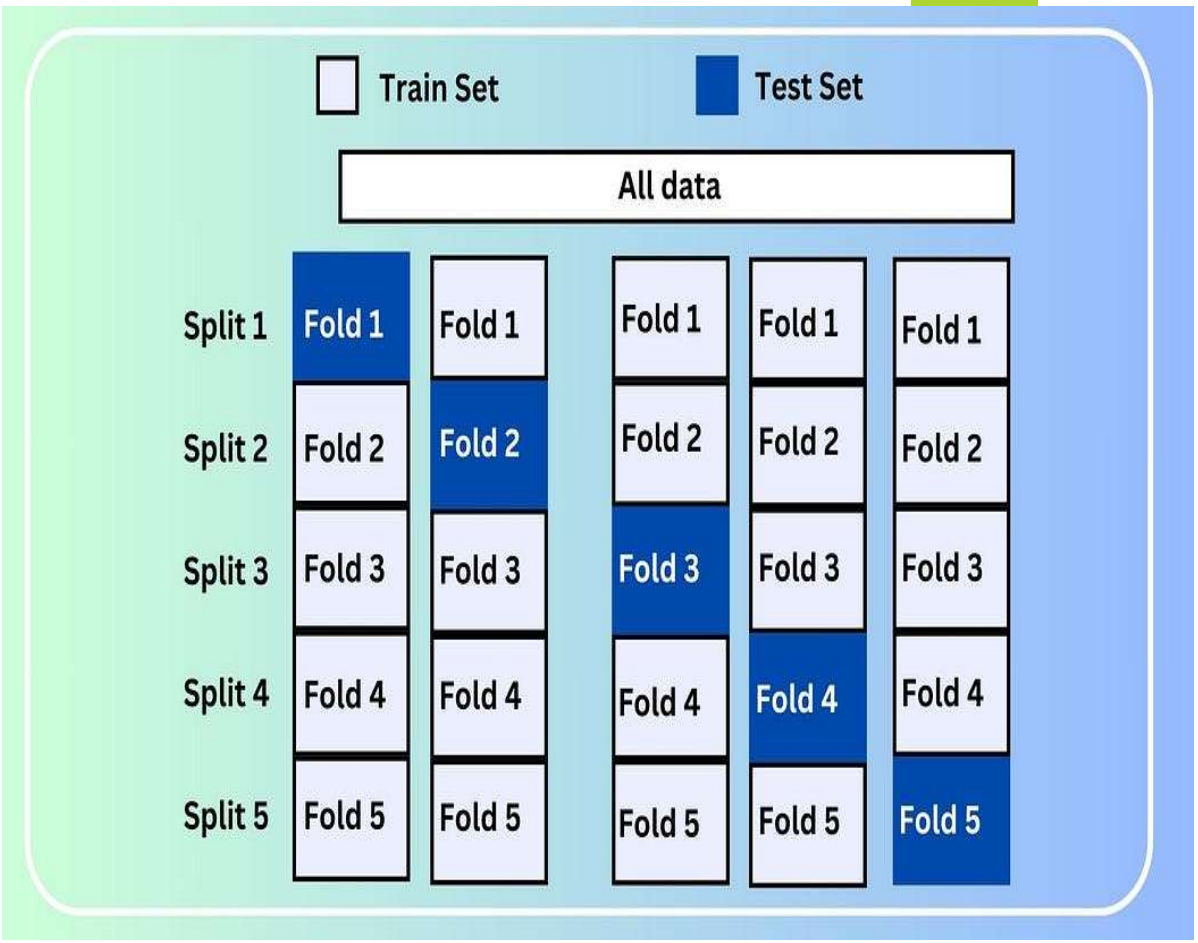


2. K-FOLD VALIDATION

K-fold validation is a method of evaluating a machine learning model by splitting the data into k folds. The model is trained k times, each time using a different fold as the validation set and the remaining folds as the training set. The performance metrics obtained from each fold are averaged to provide a more robust estimate of the model's generalization performance.




Total available labeled data
Training set
Held-out validation set
Train on this
Evaluate on this


Train Set
Test Set
All data
Split 1
Split 2
Split 3
Split 4
Split 5
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5

Evaluation Metrics	Type	Model	metrics
	Supervised learning	**Classification**	1. Classification Accuracy 2. Logarithmic loss 3. Area under Curve 4. F1 score 5. Precision 6. Recall 7. Confusion Matrix
		Regression	1. Mean Absolute Error 2. Mean Squared Error 3. Root Mean Square Error 4. Root Mean Square Logarithmic Error 5. R2 – Score
	Un supervised learning	**Clustering**	1. Within-Cluster Sum Square 2. Silhouette Coefficient 3. Calinski-Harabasz Index 4. Davies-Bouldin Index 5. Cumulative Explained Variance
		Dimensionality reduction	1. Trustworthiness 2. Sammon's Mapping
	Reinforcement learning	**Positive,Negative**	1. Cumulative Reward 2. Average Reward 3. Discounted Reward

SUPERVISED · CLASSIFICATION

Confusion Matrix

f(x) EQUATION

$$\begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix}$$

A 2×2 (or N×N) table of predicted class vs. actual class

EXPLANATION

The confusion matrix is not a single number but a table that lays out every combination of predicted vs. actual class. It is the foundation from which Accuracy, Precision, Recall, and F1 are all calculated, and it reveals exactly which classes the model confuses.

WORKED EXAMPLE

Out of 100 samples:
TP = 50, TN = 35, FP = 5, FN = 10

The matrix shows 85 correct predictions (TP+TN) and 15 errors (FP+FN) — with more errors from missed positives (FN) than false alarms (FP).

SUPERVISED · CLASSIFICATION

Classification Accuracy

f(x) EQUATION

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative

EXPLANATION

The proportion of total predictions the model got right. It is the simplest and most intuitive classification metric, but it becomes misleading when the classes are imbalanced (e.g. 95% of samples belong to one class).

WORKED EXAMPLE

A model classifies 100 test samples.
Correct predictions = 90
Incorrect predictions = 10

Accuracy = 90 / 100 = 0.90 → 90%

The model is correct 9 times out of 10.

SUPERVISED · CLASSIFICATION

Logarithmic Loss (Log Loss)

f(x) EQUATION

$$\text{LogLoss} = -(1/N) \sum [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$$

y_i = actual label (0/1), p_i = predicted probability of class 1, N = number of samples

EXPLANATION

Log Loss measures how well the predicted probabilities match the actual labels, not just whether the final class was correct. It heavily penalizes confident but wrong predictions, so a model that is "sure" and incorrect is punished more than one that is unsure.

WORKED EXAMPLE

True label y = 1, predicted probability p = 0.90

Loss = -log(0.90) = 0.105 (small – good)

If instead p = 0.10 (confidently wrong):
Loss = -log(0.10) = 2.303 (large – heavily penalized)

SUPERVISED · CLASSIFICATION

Area Under the Curve (AUC-ROC)

f(x) EQUATION

$$AUC = \int_0^1 TPR \; d(FPR)$$

TPR = True Positive Rate (Recall), FPR = False Positive Rate, integrated across all thresholds

EXPLANATION

AUC summarizes the Receiver Operating Characteristic (ROC) curve into a single number describing how well the model separates the two classes across every possible decision threshold. It ranges from 0.5 (random guessing) to 1.0 (perfect separation).

WORKED EXAMPLE

A model produces AUC = 0.85.

Interpretation: if one positive and one negative sample are picked at random, there is an 85% chance the model ranks the positive sample higher than the negative one.

SUPERVISED · CLASSIFICATION

F1 Score

f(x) EQUATION

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

The harmonic mean of Precision and Recall

EXPLANATION

F1 balances Precision and Recall into one number. It is especially useful with imbalanced classes, where Accuracy can look good even if the model performs poorly on the minority class, because F1 punishes models that sacrifice one metric to boost the other.

WORKED EXAMPLE

Precision = 0.80, Recall = 0.60

F1 = 2 × (0.80 × 0.60) / (0.80 + 0.60)
= 2 × 0.48 / 1.40
= 0.686 → 68.6%

SUPERVISED · CLASSIFICATION

Precision

f(x) EQUATION

$$\text{Precision} = TP / (TP + FP)$$

"Of everything predicted positive, how much was actually positive?"

EXPLANATION

Precision measures how trustworthy a positive prediction is. High precision is critical when false positives are costly – for example, flagging a healthy patient as sick, or triggering an unnecessary equipment shutdown.

WORKED EXAMPLE

Model predicts 50 samples as positive.
40 are truly positive (TP), 10 are false alarms (FP).

Precision = 40 / (40 + 10) = 40/50 = 0.80 → 80%

SUPERVISED · CLASSIFICATION

Recall (Sensitivity)

f(x) EQUATION

$$\text{Recall} = TP / (TP + FN)$$

"Of all the actual positives, how many did the model catch?"

EXPLANATION

Recall measures how completely the model finds the positive class. High recall matters when missing a positive case is costly – for example, missing a fraud transaction or missing a fault in an electrical grid.

WORKED EXAMPLE

There are 60 actual positive samples.
Model correctly identifies 40 (TP); misses 20 (FN).

Recall = 40 / (40 + 20) = 40/60 = 0.667 → 66.7%

SECTION 2

Supervised Learning

Regression Models

MAE | MSE | RMSE | RMSLE | R^2 Score

SUPERVISED · REGRESSION

Mean Absolute Error (MAE)

f(x) EQUATION

$$MAE = (1/n) \Sigma |y_i - \hat{y}_i|$$

y_i = actual value, $\hat{y}_i$ = predicted value, n = number of samples

EXPLANATION

MAE is the average of the absolute differences between actual and predicted values. It is easy to interpret because it stays in the same unit as the target (e.g. MW for load forecasting), and it treats every error equally regardless of size.

WORKED EXAMPLE

Actual vs predicted errors: 2, -3, 1
Absolute errors: 2, 3, 1

MAE = (2 + 3 + 1) / 3 = 6/3 = 2.0

On average, predictions are off by 2 units.

SUPERVISED · REGRESSION

Mean Squared Error (MSE)

f(x) EQUATION

$$MSE = (1/n) \Sigma (y_i - \hat{y}_i)^2$$

Squares each error before averaging

EXPLANATION

MSE squares each error before averaging, so larger errors are penalized much more heavily than small ones. This makes it sensitive to outliers and useful when large mistakes are especially undesirable, but its unit is squared and less interpretable.

WORKED EXAMPLE

Same errors: 2, -3, 1
Squared errors: 4, 9, 1

MSE = (4 + 9 + 1) / 3 = 14/3 = 4.67

SUPERVISED · REGRESSION

Root Mean Square Error (RMSE)

f(x) EQUATION

$$RMSE = \sqrt{MSE} = \sqrt{[(1/n)\ \Sigma\ (y_i - \hat{y}_i)^2]}$$

Square root of MSE brings the metric back into the original unit

EXPLANATION

RMSE takes the square root of MSE, returning the error to the same unit as the target variable while still penalizing large errors more than small ones. It is the most widely reported error metric in load-forecasting research because it is directly comparable to the measured quantity (e.g. MW).

WORKED EXAMPLE

MSE = 4.67 (from previous slide)

RMSE = √4.67 = 2.16

Example from real STLF work: an LSTM model with lookback = 48 and top-10 features achieved RMSE = 178.07 MW, a ~59.7% improvement over the operator's baseline forecast.

SUPERVISED · REGRESSION

Root Mean Square Logarithmic Error (RMSLE)

f(x) EQUATION

$$\text{RMSLE} = \sqrt{\left[(1/n)\ \Sigma\ (\log(y_i+1) - \log(\hat{y}_i+1))^2\right]}$$

Uses log(value + 1) so zero values remain defined

EXPLANATION

RMSLE compares the log of actual vs. predicted values, which compresses large-magnitude errors and penalizes under-prediction more than over-prediction. It is useful when target values span several orders of magnitude or when relative error matters more than absolute error.

WORKED EXAMPLE

Actual y = 100, predicted ŷ = 90

log(101) – log(91) = 4.615 – 4.511 = 0.104
Squared = 0.0108

Averaged over all samples and square-rooted gives the final RMSLE value.

SUPERVISED · REGRESSION

R^2 Score (Coefficient of Determination)

f(x)

EQUATION

$$R^2 = 1 - [\ \Sigma(y_i - \hat{y}_i)^2 \ / \ \Sigma(y_i - \bar{y})^2\ ]$$

$\bar{y}$ = mean of actual values; numerator = residual variance, denominator = total variance

EXPLANATION

R^2 expresses how much of the variance in the target variable is explained by the model, relative to simply predicting the mean every time. A value of 1.0 means a perfect fit; a value of 0 means the model is no better than the mean.

WORKED EXAMPLE

An LSTM STLF model (lookback = 48, top-10 features) achieved:

$R^2 = 0.9829$

This means the model explains 98.29% of the variance in the actual demand – an excellent fit.

SECTION 3

Unsupervised Learning

Clustering Models

WCSS | Silhouette | Calinski-Harabasz | Davies-Bouldin | Cum. Explained Variance

Within-Cluster Sum of Squares (WCSS)

f(x) EQUATION

$$WCSS = \Sigma_k \Sigma_{i \in C_k} || x_i - \mu_k ||^2$$

μ_k = centroid of cluster k; measures compactness of each cluster

EXPLANATION

WCSS sums the squared distance between each point and its own cluster centroid. Lower WCSS means tighter, more compact clusters. It decreases as the number of clusters k increases, so it is mainly used with the “elbow method” to choose a good value of k.

WORKED EXAMPLE

Cluster with 3 points at distances 1, 2, 1 from its centroid:

WCSS = $1^2 + 2^2 + 1^2 = 1 + 4 + 1 = 6$

Summing this across all k clusters gives the total WCSS for that clustering solution.

UNSUPERVISED · CLUSTERING

Silhouette Coefficient

f(x)

EQUATION

$$s(i) = (b(i) - a(i)) / max(a(i), b(i))$$

a(i) = mean distance to own cluster, b(i) = mean distance to nearest other cluster

i

EXPLANATION

The Silhouette Coefficient scores how well each point fits its assigned cluster compared to the next-nearest cluster. Values range from -1 to +1: values near +1 mean well-clustered points, near 0 mean borderline points, and negative values suggest a point may be in the wrong cluster.

✓

WORKED EXAMPLE

Point i has a(i) = 0.2 (close to own cluster) and b(i) = 0.8 (far from nearest other cluster).

s(i) = (0.8 – 0.2) / max(0.2, 0.8)
= 0.6 / 0.8 = 0.75 → well-clustered point

SECTION 4

Unsupervised Learning

Dimensionality Reduction

Trustworthiness

Sammon's Mapping

UNSUPERVISED · DIMENSIONALITY REDUCTION

Trustworthiness

f(x)

EQUATION

$$T(k) = 1 - \left[\frac{2}{nk(2n-3k-1)} \right] \sum_i \sum_{j \in U_k(i)} \left(r(i,j) - k \right)$$

$U_k(i)$ = points that are "false neighbors" after projection; r(i,j) = rank of j among i's original neighbors

EXPLANATION

Trustworthiness checks whether points that appear close together in the low-dimensional (reduced) space were actually close together in the original high-dimensional space. It penalizes the projection for introducing false neighbors that were not truly similar.

WORKED EXAMPLE

A 2D embedding scores Trustworthiness = 0.95

Interpretation: 95% of the nearest neighbors shown in the 2D visualization are genuine neighbors from the original high-dimensional data – only 5% are misleading artifacts.

SECTION 5

Reinforcement Learning

Positive & Negative Reward Signals

Cumulative Reward

Average Reward

Discounted Reward

REINFORCEMENT LEARNING

Cumulative Reward

f(x)

EQUATION

$$G = \Sigma_{t=0}^{T} r_t$$

r_t = reward received at time step t; T = final time step of the episode

EXPLANATION

Cumulative Reward is the simple total of all rewards an agent collects over one episode. It is the most direct measure of how well an agent performed, though it does not distinguish between rewards received early versus late in the episode.

WORKED EXAMPLE

Rewards per step: 1, 0, 2, -1, 3

Cumulative Reward = 1 + 0 + 2 + (-1) + 3
= 5

The agent finished the episode with a net reward of 5.

REINFORCEMENT LEARNING

Average Reward

f(x)

EQUATION

$$\bar{R} = (1/T) \sum_{t=0}^{T} r_t$$

T = number of time steps in the episode

EXPLANATION

Average Reward divides cumulative reward by the number of steps taken, giving reward-per-step. It is especially useful for comparing agents across episodes of different lengths, or for continuing (non-episodic) tasks where total reward alone is not meaningful.

WORKED EXAMPLE

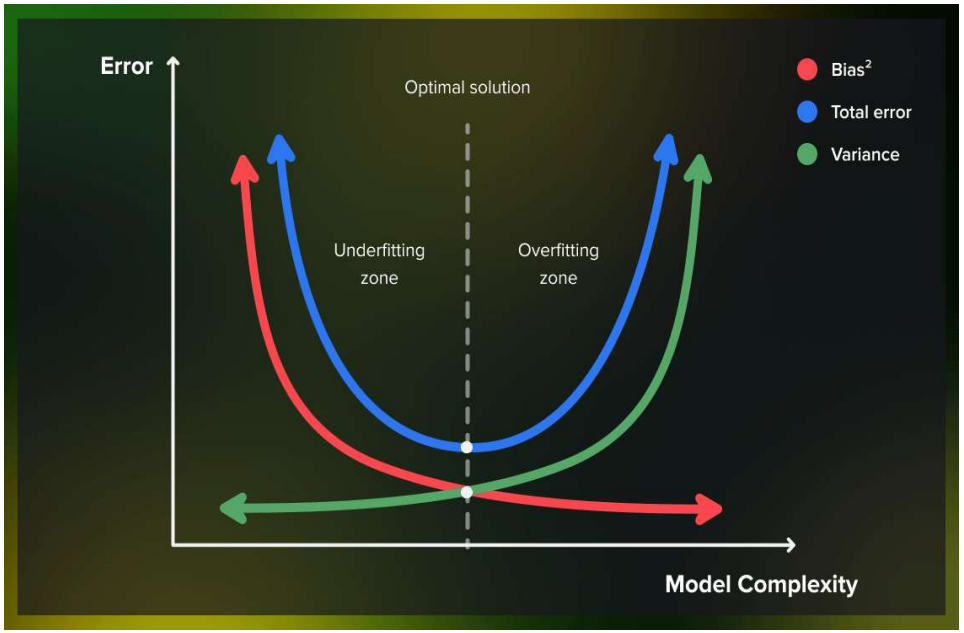
Cumulative Reward = 5 over 5 time steps

Average Reward = 5 / 5 = 1.0

On average, the agent earns 1.0 reward per time step.

Overfitting and underfitting

Overfitting and underfitting are two common problems that can occur in machine learning. Overfitting occurs when a model learns the training data too well and starts to memorize the noise and outliers in the data. This can lead to the model performing poorly on new data that it has not seen before. Underfitting occurs when a model does not learn the training data well enough and is unable to make accurate predictions.



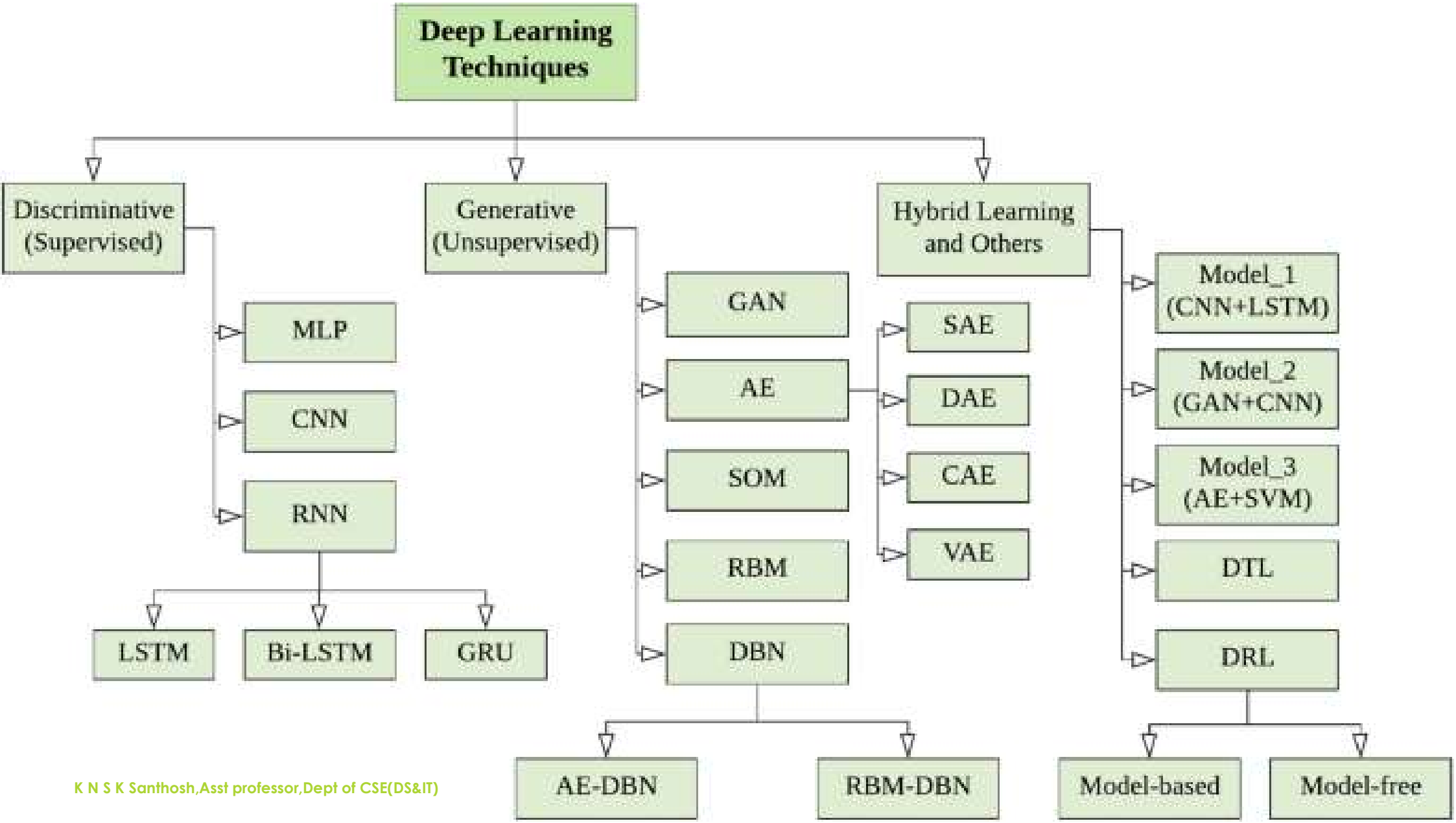
Optimization and generalization

- Optimization refers to the process of finding **the best parameters for a model**, while generalization refers to the **ability of a model to make accurate predictions on new data**.
- The goal of machine learning is to build models that can generalize well to new data. However, there is a tension between optimization and generalization. If we **optimize** a model too much, it may become too complex and start to **overfit** the training data. This means that the model will make accurate predictions on the training data, but it will not generalize well to new data.
- On the other hand, if we do not optimize the model enough, it may not learn the training data well enough and will not be able to make accurate predictions on new data.
- The challenge in machine learning is to find a balance between optimization and generalization. This can be done by using regularization techniques, which add a penalty to the model's complexity.

Regularization

Regularization can help to prevent the model from overfitting the training data and improve its ability to generalize to new data

1. Reducing the network's size
2. Adding weight regularization
3. Adding dropout


Deep Learning Techniques
Discriminative (Supervised)
Generative (Unsupervised)
Hybrid Learning and Others
MLP
CNN
RNN
LSTM
Bi-LSTM
GRU
GAN
AE
SOM
RBM
DBN
SAE
DAE
CAE
VAE
AE-DBN
RBM-DBN
Model_1 (CNN+LSTM)
Model_2 (GAN+CNN)
Model_3 (AE+SVM)
DTL
DRL
Model-based
Model-free